Clarion: A Local, Ambiguity-Aware Conversational Database Assistant

React 19 SPA → Vite middleware API → Python worker → Qwen2.5-Coder-1.5B GGUF / llama.cpp

Architecture and method report · 6 November 2026 · Prototype status, not a production-readiness claim

Project goal. Clarion lets users work with uploaded tabular data through natural language. It routes metadata questions, read queries, and explicit write requests through separate paths; the intended ambiguity layer combines a lightweight classifier with an SLM analyst, while write requests use a preview-and-confirm workflow.

Architecture at a glance

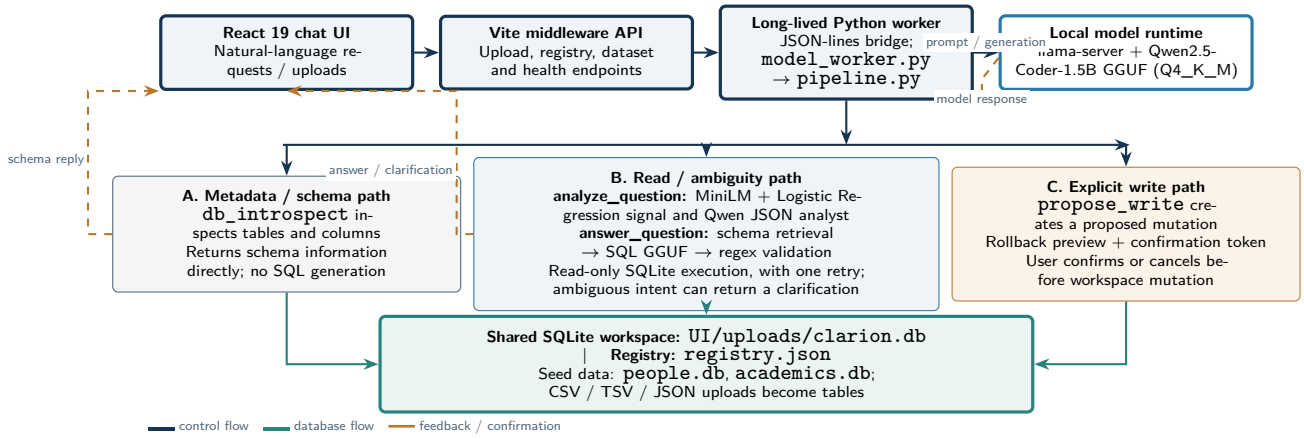


Figure 1. Current Clarion runtime and its three request paths. The figure describes the supplied project audit; it does not imply that every component is production-hardened.

What happens to a request

1. **Ingest and register.** CSV, TSV, and JSON uploads are converted into tables in a shared SQLite workspace; a registry tracks available datasets. The project also seeds the workspace from local SQLite fixtures.
2. **Route by request type.** Schema/metadata questions use `db_introspect`. Ordinary questions enter `analyze_question`. Explicit mutations enter `propose_write`.
3. **Read and clarify.** The read path combines a MiniLM/Logistic Regression classifier signal with a Qwen JSON analysis, retrieves schema context, generates SQL, validates it, and executes against a read-only SQLite connection. A clarification is returned when the intent is not sufficiently resolved.
4. **Preview writes.** Write requests are proposed first, previewed through a rollback flow, and associated with a confirmation token. The user can confirm or cancel before the mutation is applied.

Capability	Current implementation detail
Data inputs	CSV / TSV / JSON ingestion into a SQLite workspace; seeded SQLite datasets are also present.
Read path	Question analysis, schema retrieval, SQL generation, regex validation, read-only execution, and one retry.
Write path	Explicit write proposal, rollback preview, token-based confirm/cancel flow.
Runtime	React 19, Vite middleware API, persistent Python worker, llama.cpp server, Qwen2.5-Coder-1.5B Q4_K_M model.

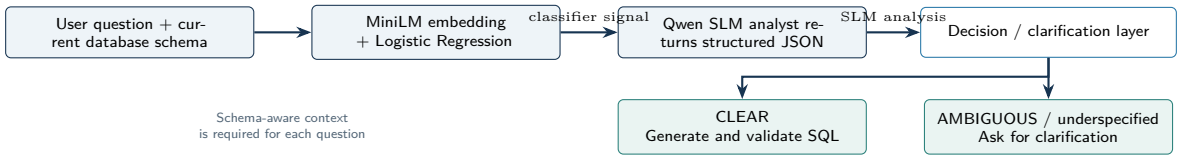
Research framing. Ambiguity detection and clarification are established research topics. Clarion's testable question is whether a small, schema-conditioned classifier can selectively coordinate with a local SLM to improve the accuracy–latency–resource trade-off across unseen user databases.
Evaluation focus. Macro-F1, per-class recall, false-clarification rate, clarification resolution, SQL execution accuracy, cross-schema generalization, end-to-end latency, and peak RAM.

Method, Evaluation and Current Status

A candid snapshot of what is implemented, what is measured, and what remains to be validated.

Ambiguity-aware decision method

The intended method is to use a lightweight discriminative model for a first-pass signal and a compact language model for contextual interpretation. The database schema is part of the input: the same phrase can be clear in one schema and ambiguous in another.



Important implementation caveat. The current fusion routine is not yet a genuinely balanced hybrid: when both classifier and SLM outputs are present, the selected decision follows the SLM label in the audited branches. The classifier therefore does not currently act as an effective veto or calibrated gate. The diagram above shows the intended component roles, not a claim that the fusion policy is already correct.

Measured prototype status

Measure	Observed value	Interpretation / limitation
Classifier	525 examples; 55.45% held-out; 54.2% unseen-schema	Weak performance; current model is not reliable enough to act as a standalone ambiguity gate.
Automated tests	26 passed / 5 failed	The audited suite is red; safety-path tests need to be restored and rerun.
End-to-end SQL accuracy	Not established	The prior 30-pair score used a mismatched model/prompt path and inconsistent grading; do not quote it as a validated headline result.
Latency / peak RAM	Not established here	Measure the full served pipeline on fixed hardware; generation-only timing is insufficient.

Current limitations and next engineering steps

Priority	Required improvement
P0 – decision quality	Fix classifier/SLM fusion; put the question before schema text; prevent schema truncation; calibrate thresholds on held-out data.
P0 – correctness	Restore failing tests; evaluate the exact served model with the production schema builder and documented result normalization; expand database-level holdouts.
P1 – SQL safety	Replace regex-only validation with SQLGlot parsing, one-statement enforcement, table/column checks, and read/write allowlists. Keep read-only SQLite and an authorizer as defense in depth.
P1 – performance	Cache schema metadata; avoid repeated per-column value scans; bound prompt context; measure end-to-end latency and resource use.
P1 – write UX / deployment	Show before/after diffs and re-check preview state at confirmation; separate the API from Vite dev mode before LAN use; add auth/origin checks, WAL/busy timeout, a write queue, and process shutdown.

Selected related work

Ambiguity benchmark. A. Bhaskar, T. Tomar, A. Sathe, and S. Sarawagi, "Benchmarking and Improving Text-to-SQL Generation under Ambiguity," EMNLP 2023, pp. 7053–7074.
https://aclanthology.org/2023.emnlp-main.436/

Conversational ambiguity. M. Dong et al., "PRACTIQ: A Practical Conversational Text-to-SQL Dataset with Ambiguous and Unanswerable Queries," NAACL 2025, pp. 255–273.
https://aclanthology.org/2025.naacl-long.13/

Interactive disambiguation. "CLEAR: A Parser-Independent Disambiguation Framework for NL2SQL," IEEE ICDE 2025, DOI: 10.1109/ICDE65448.2025.00247.
https://ieeexplore.ieee.org/document/11113237/

Execution-grounded intent probing. Y. Yin et al., "Probing Ambiguity: Adversarial Intent Resolution for Robust Text-to-SQL," EMNLP 2026.
https://aclanthology.org/2026.emnlp-main.2446/